Docker Compose. Многоконтейнерные приложения

Описываем Nginx, Express API и PostgreSQL в одном compose.yaml

Цель лекции

- Понимать назначение Docker Compose и декларативного описания сервисов.
- Разбирать YAML и структуру compose.yaml.
- Конфигурировать services, build, image, ports, environment, networks и volumes.
- Запускать и диагностировать стек через docker compose.
- Различать остановку, удаление контейнеров и удаление данных тома.

Что уже умеем

- Создавать Dockerfile для Express API и собирать образ.
- Запускать PostgreSQL 17 с постоянным Volume.
- Создавать пользовательскую bridge-сеть и обращаться к БД по DNS-имени.
- На предыдущей лекции запускали компоненты отдельными docker run; теперь опишем целое приложение.

Учебные вопросы

№	Учебный вопрос	Практический результат
1	Назначение Compose	Описать приложение как набор сервисов
2	Синтаксис compose.yaml	Прочитать и составить YAML
3	Сети, тома и конфигурация	Соединить API и PostgreSQL
4	Команды Compose	Запустить и проверить стек
5	Nginx + Express + PostgreSQL	Настроить обратный прокси

Назначение Docker Compose

От отдельных команд к описанию всего приложения

Проблема ручного запуска

- Три docker run требуют указания многих параметров и порядка действий.
- Каждый запуск вручную должен учитывать сети, тома, переменные и порты.
- В команде сложно передать все параметры и воспроизвести среду на другом ПК.
- Compose описывает желаемое состояние приложения в YAML-файле.

Что такое Docker Compose

- Docker Compose — инструмент определения и управления многоконтейнерными приложениями.
- Описывает сервисы, конфигурацию, сети и тома в compose.yaml.
- Современная команда: docker compose (пробел), плагин Docker CLI.
- Обычный Docker Engine остаётся средой выполнения контейнеров.

От docker run к Docker Compose

Как перейти от ручного запуска контейнеров к описанию многоконтейнерного приложения в одном файле

Ручной запуск: docker run

```
# 1. Nginx
$ docker run -d \
    --name nginx \
    -p 80:80 \
    --network app-net \
    -v ./nginx/nginx.conf:/etc/nginx/nginx.conf \
    nginx:latest
```

Nginx

```
# 2. Express API
$ docker run -d \
    --name api \
    -p 3000:3000 \
    --network app-net \
    -e NODE_ENV=production \
    -v ./api:/app \
    node:18
```

Express API

```
# 3. PostgreSQL
$ docker run -d \
    --name db \
    -p 5432:5432 \
    --network app-net \
    -e POSTGRES_DB=app \
    -e POSTGRES_USER=app \
    -e POSTGRES_PASSWORD=secret \
    -v pgdata:/var/lib/postgresql/data \
    postgres:16
```

PostgreSQL

- Много длинных команд
- Нужно вручную создать сеть
- Нужно вручную подключать тома
- Легко ошибиться в параметрах
- Сложнее запускать и останавливать всё приложение

Переход к декларативному описанию

Docker Compose

compose.yaml

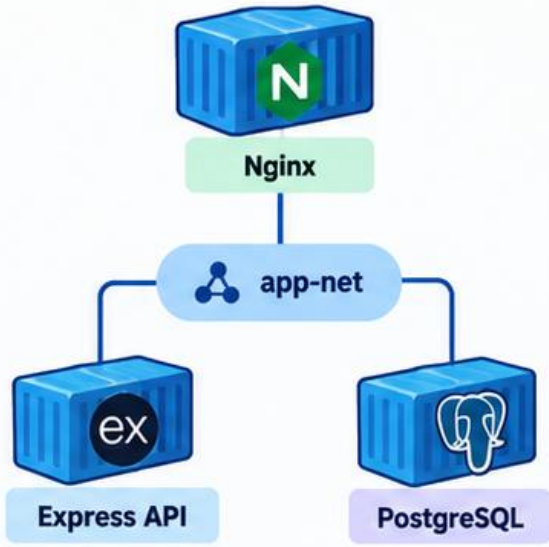
```yaml
services:
  nginx:
    image: nginx:latest
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf
    networks:
      - app-net

  api:
    image: node:18
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
    volumes:
      - ./api:/app
    networks:
      - app-net

  db:
    image: postgres:16
    ports:
      - "5432:5432"
    environment:
      - POSTGRES_DB=app
      - POSTGRES_USER=app
      - POSTGRES_PASSWORD=secret
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - app-net

networks:
  app-net:
    driver: bridge

volumes:
  pgdata:
```



- Один файл описывает всё приложение
- Сервисы запускаются одной командой
- Сети и тома создаются автоматически
- Удобно смотреть логи и перезапускать сервисы
- Проще сопровождать многоконтейнерный проект

Ключевые команды

Запуск всех сервисов	Просмотр статуса сервисов	Просмотр логов	Остановка и удаление
`$ docker compose up -d`	`$ docker compose ps`	`$ docker compose logs`	`$ docker compose down`

Итог: Docker Compose упрощает запуск, настройку и сопровождение многоконтейнерных приложений.

Ручные команды и Compose

docker run

- Отдельная команда для каждого контейнера.
- Создание сети и томов часто отдельно.
- Труднее повторить одни и те же настройки.
- Удобен для одиночных экспериментов.

docker compose

- Один конфигурационный файл для нескольких сервисов.
- Создаёт сети и тома проекта по декларации.
- Команды up, down, logs управляют всем стеком.
- Удобен для воспроизводимых учебных проектов.

Объекты в терминологии Compose

- Project — группа ресурсов, управляемых Compose.
- Service — описание того, как создаются однотипные контейнеры приложения.
- Container — запущенный экземпляр сервиса.
- Network — среда общения сервисов; Volume — постоянное хранилище.
- Имя проекта обычно зависит от каталога, но может задаваться явно.

Проверка Compose

BASH

```
docker compose version
sudo docker compose version
```

Что важно

- Плагин Compose должен быть установлен.
- Команды Docker в Debian выполняем с sudo; конфигурация и версия читаются без создания контейнеров.
- Не используем старую команду docker-compose в учебных примерах.

Файл compose.yaml и YAML

Отступы определяют структуру конфигурации

Что такое YAML

- YAML — текстовый формат описания структурированных данных.
- Вложенность задаётся пробелами, не фигурными скобками.
- Для записи списков используют дефис; ключ отделяется двоеточием.
- Отступы должны быть согласованы; табуляция для отступов не подходит.

Минимальный compose.yaml

```yaml
services:
  web:
    image: nginx:stable
    ports:
      - "8080:80"
```

Что важно

- services — верхний уровень описания сервисов.
- web — имя сервиса; image задаёт готовый образ.
- ports публикует 8080 хоста на 80 контейнера.
- Кавычки вокруг сопоставления портов исключают ошибочную интерпретацию YAML.

Основные ключи конфигурации

- services описывает контейнерные сервисы.
- image — готовый образ, build — каталог и контекст сборки Dockerfile.
- ports — публикация порта на хосте; environment — переменные внутри контейнера.
- volumes — монтирование хранилищ; networks — подключение к сетям.
- depends_on — зависимости порядка запуска и, с условием health, готовности.

Разбор build и image

```yaml
services:
  api:
    build: ./api
    image: course-api:7
  db:
    image: postgres:17
```

Что важно

- api собирается из Dockerfile в каталоге ./api и получает тег course-api:7.
- db запускается из готового образа postgres:17.
- Код API и PostgreSQL не объединяются в один контейнер.

Почему файл называется compose.yaml

- Compose поддерживает compose.yaml и другие допустимые имена файла.
- В курсе используем compose.yaml как основной вариант.
- Старая верхнеуровневая секция version для современной Compose Specification не требуется.
- Файл хранится в корне каталога учебного проекта.

Проверка синтаксиса до запуска

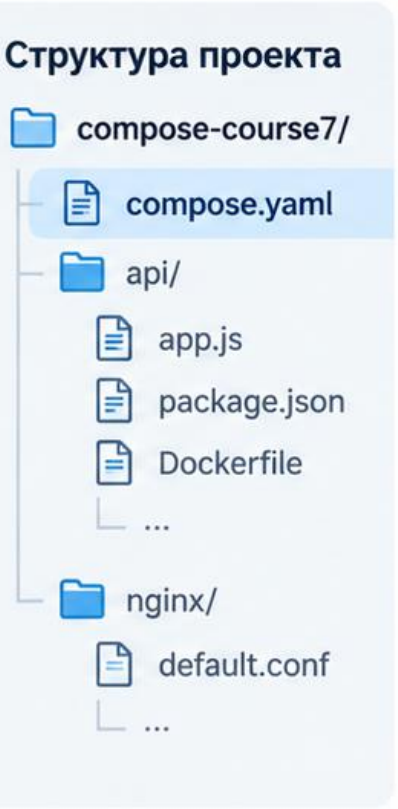
```bash
sudo docker compose config
sudo docker compose config --services
```

Что важно

- config показывает итоговую конфигурацию и помогает находить ошибки YAML.
- --services выводит имена сервисов.
- Проверка не требует запущенных контейнеров.

Структура compose.yaml — services, networks, volumes

Структура проекта



compose.yaml — YAML

```yaml
version: '3.9'
services:
  nginx:
    image: nginx:latest
    ports:
      - "8080:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - api
  api:
    build: ./api
    ports:
      - "3000:3000"
    environment:
      DB_HOST: db
      DB_PORT: 5432
    depends_on:
      - db
  db:
    image: postgres:17
    environment:
      POSTGRES_DB: shop
      POSTGRES_USER: shopuser
      POSTGRES_PASSWORD: shoppass
    volumes:
      - pgdata:/var/lib/postgresql/data

networks:
  app-network:
    driver: bridge

volumes:
  pgdata:
```

1 services — контейнеры (сервисы)

Определяют, какие контейнеры запускать и как их настраивать.

nginx
- image: nginx:latest
- ports: 8080:80
- volumes: ./nginx/default.conf
- depends_on: api

api
- build: ./api
- ports: 3000:3000
- environment: DB_HOST=db
- depends_on: db

db
- image: postgres:17
- environment: POSTGRES_DB, POSTGRES_USER, ...
- volumes: pgdata:/var/lib/postgresql/data

2 networks — сети

Создают пользовательскую сеть для всех сервисов.

app-network
- драйвер: bridge
- все сервисы в одной сети
- встроенный DNS (имена сервисов)

3 volumes — тома

Создают именованные тома для постоянного хранения данных.

pgdata
- хранит данные PostgreSQL
- не удаляется при docker compose down
- удаляется при docker compose down -v

Сети, тома и переменные

Как сервисы взаимодействуют, а данные сохраняются

Сеть Compose по умолчанию

- Если явно не заданы networks, Compose создаёт сеть проекта default.
- Контейнеры внутри сети обращаются друг к другу по именам сервисов.
- API подключается к БД по имени db и порту 5432.
- IP-адреса назначаются динамически; в коде не используем фиксированные IP.

Ссылка на базу по имени сервиса

```yaml
services:
  api:
    build: ./api
    environment:
      PGHOST: db
      PGPORT: "5432"
  db:
    image: postgres:17
```

Что важно

- PGHOST=db — имя сервиса Compose, встроенный DNS определяет его адрес.
- localhost внутри API означал бы сам API-контейнер.
- Публикация порта 5432 на хосте для связи API → db не нужна.

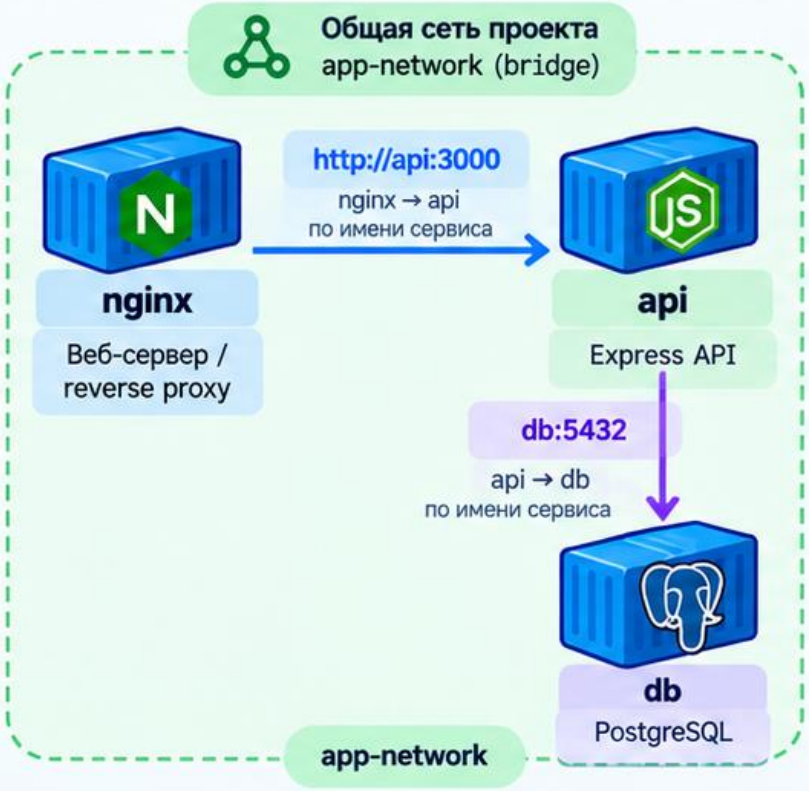
Сервисы Compose и встроенный DNS

Как контейнеры в Docker Compose находят друг друга по именам сервисов внутри общей сети

compose.yaml

```yaml
version: '3.9'
services:
  nginx:
    image: nginx:latest
    ports:
      - "8080:80"
    depends_on:
      - api
  api:
    build: ./api
    ports:
      - "3000:3000"
    depends_on:
      - db
  db:
    image: postgres:17
    environment:
      POSTGRES_DB: shop
      POSTGRES_USER: shopuser
      POSTGRES_PASSWORD: shoppass
networks:
  app-network:
    driver: bridge
```

Сеть проекта Docker Compose



Как работает встроенный DNS

1. Compose создаёт общую сеть проекта

При запуске docker compose up создаётся пользовательская сеть (например, app-network), к которой подключаются все сервисы.

2. Каждый сервис получает DNS-имя

Встроенный DNS автоматически создаёт DNS-записи для сервисов. Имя сервиса совпадает с DNS-именем.

- DNS-имя: nginx
- DNS-имя: api
- DNS-имя: db

3. Контейнер api обращается к db

Приложение в контейнере api подключается к базе данных по имени db и порту 5432: например, postgres://db:5432.

db:5432

4. Контейнер nginx обращается к api

Nginx проксирует запросы на сервис api по имени api и порту 3000: например, proxy_pass http://api:3000;

api:3000

Правильно

- DB_HOST=db
- proxy_pass http://api:3000;
- Контейнеры общаются по именам сервисов

Используйте имена сервисов — они разрешаются через встроенный DNS.

Неправильно

- DB_HOST=localhost
- proxy_pass http://localhost:3000;
- localhost внутри контейнера — это сам контейнер

localhost ссылается на текущий контейнер, а не на другой сервис Compose.

Что важно понять

1. Сервисы Compose по умолчанию находятся в общей сети проекта.
2. Имя сервиса работает как DNS-имя.
3. Для связи между контейнерами публикация порта не обязательна.
4. Для доступа с хоста используют опубликованные порты, а не внутренние DNS-имена.

Постоянное хранение PostgreSQL

- Без вынесенных данных пересоздание контейнера удалит его writable layer.
- Именованный том можно подключить к каталогу БД.
- Для postgres:17 каталог данных /var/lib/postgresql/data.
- docker compose down не удаляет именованные тома без флага -v.
- Нельзя одновременно запускать два сервера PostgreSQL с одним каталогом данных.

Описание именованного тома

```yaml
services:
  db:
    image: postgres:17
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Что важно

- Верхнеуровневый volumes объявляет логический именованный том проекта.
- Монтирование сохраняет файлы PostgreSQL вне writable layer контейнера.
- Том создаётся и управляется Compose.

Переменные окружения

- environment передаёт параметры контейнеру.
- Для учебной БД можно указать фиксированные значения без хранения рабочего пароля.
- В реальной среде секреты не следует публиковать в git и открытых конфигурациях.
- Изучение полноценного управления секретами запланировано на следующую тему.

Параметры и healthcheck PostgreSQL

```yaml
environment:
  POSTGRES_DB: shop
  POSTGRES_USER: student
  POSTGRES_PASSWORD: lab-demo-only
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U student -d shop"]
  interval: 5s
  timeout: 3s
  retries: 10
```

Что важно

- POSTGRES_* используются при первичной инициализации пустого тома.
- pg_isready проверяет готовность сервера принимать соединения.
- Проверка healthcheck полезна для зависимых сервисов.

depends_on и готовность сервиса

- Краткая форма depends_on устанавливает порядок запуска, но сама по себе не ждёт готовности БД.
- В длинной форме condition: service_healthy ожидает успешного healthcheck.
- Даже с healthcheck приложение должно корректно обрабатывать последующие сбои БД.
- Рассматриваем готовность как практическую проблему, а не как гарантию вечной доступности.

Ожидание готовности DB

```yaml
api:
  depends_on:
    db:
      condition: service_healthy
```

Что важно

- API запускается после того, как сервис db станет healthy.
- Условие зависит от корректно настроенного healthcheck у db.

Команды управления Compose

Запуск, диагностика и корректное завершение

Основные команды Compose

Команда	Назначение
docker compose up -d	Создать и запустить сервисы в фоне
docker compose up -d --build	Пересобрать образ при запуске
docker compose ps	Состояние сервисов
docker compose logs -f api	Журналы сервиса api
docker compose exec db sh	Команда внутри контейнера db
docker compose stop	Остановить сервисы без удаления
docker compose start	Запустить ранее остановленные
docker compose down	Удалить контейнеры и сети проекта
docker compose down -v	Дополнительно удалить объявленные именованные тома проекта

Запуск и проверка

BASH

```
sudo docker compose config
sudo docker compose up -d --build
sudo docker compose ps
```

Что важно

- Сначала проверяем конфигурацию.
- Флаг --build собирает API по Dockerfile.
- ps покажет созданные контейнеры и опубликованные порты.

Просмотр логов и выполнение команды

```bash
sudo docker compose logs --tail=30 api
sudo docker compose logs --tail=30 db
sudo docker compose exec db pg_isready -U student -d
```

Что важно

- logs помогает диагностировать сетевые и программные ошибки.
- exec запускает команду внутри работающего сервиса.
- Система может запускать несколько контейнеров, но обращаемся по имени сервиса.

Жизненный цикл приложения Docker Compose

1 Подготовка проекта

Создаём структуру проекта и файл compose.yaml (и другие файлы).

```
my-app/
  compose.yaml
  api/
    Dockerfile
    app.js
  nginx/
    default.conf
  db/
  ...
```

2 Запуск приложения

Создаём и запускаем контейнеры всех сервисов.

```
$ docker compose up -d
```

Создаются и запускаются контейнеры:

- **nginx** порт 8080 → 80
- **api** порт 3000
- **db** порт 5432

3 Проверка работы

Проверяем статус, логи и работу приложения.

```
$ docker compose ps
NAME          STATUS    PORTS
myapp-nginx   running   0.0.0.0:8080->80/tcp
myapp-api     running   3000/tcp
myapp-db      running   5432/tcp
```

```
$ docker compose logs -f api
```

```
$ curl http://localhost:8080/api/db-check
```

```
{
  "ok": true,
  "connected": 1
}
```

4 Изменение кода и конфигурации

Вносим изменения в код, compose.yaml или конфиги.

- app.js
- compose.yaml
- default.conf

Применяем изменения

Если изменился только код:

```
$ docker compose up -d
```

Если изменились конфигурации или сервисы:

```
$ docker compose down
$ docker compose up -d
```

5 Остановка приложения

Останавливаем и удаляем контейнеры (данные в томах сохраняются).

```
$ docker compose down
```

- **nginx** контейнер удалён
- **api** контейнер удалён
- **db** контейнер удалён

6 Полная очистка (при необходимости)

Удаляем контейнеры, сети и тома (данные в БД будут удалены).

```
$ docker compose down -v
```

Удаляется всё

- контейнеры
- сети
- тома
- данные в БД

Вносим изменения и повторяем цикл

Состояние контейнеров

Создание
`docker compose up`
Создаются контейнеры по описанию в compose.yaml

→ **Запуск**
контейнеры переходят в состояние `running` и запускаются сервисы

→ **Работа**
приложение доступно и обрабатывает запросы

→ **Остановка**
`docker compose down`
контейнеры останавливаются и удаляются

→ **Удаление**
`docker compose down -v`
удаляются контейнеры, сети и тома (данные удаляются)

Том с данными (например, БД)

pgdata

Сохраняется при docker compose down
Данные остаются на диске

Удаляется только при docker compose down -v
Все данные в томе будут удалены

stop, down и down -v

stop / start

- Контейнеры остаются существовать.
- Данные томов не удаляются.
- Можно продолжить работу без пересоздания.

down / down -v

- down удаляет контейнеры и сети проекта.
- Именованные тома сохраняются при обычном down.
- down -v удаляет объявленные именованные тома — опасно для данных.

Обновление приложения

- При изменении кода встраиваемого в образ API требуется новая сборка.
- docker compose up -d --build пересобирает API и при необходимости пересоздаёт сервис.
- Именованный том базы остаётся и используется новым контейнером.
- Проверяем не только статус контейнеров, но и HTTP-ответ маршрута.

Три сервиса: Nginx, Express API, PostgreSQL

Сквозной учебный проект приобретает единую точку HTTP-доступа

Docker Compose — Nginx → Express API → PostgreSQL

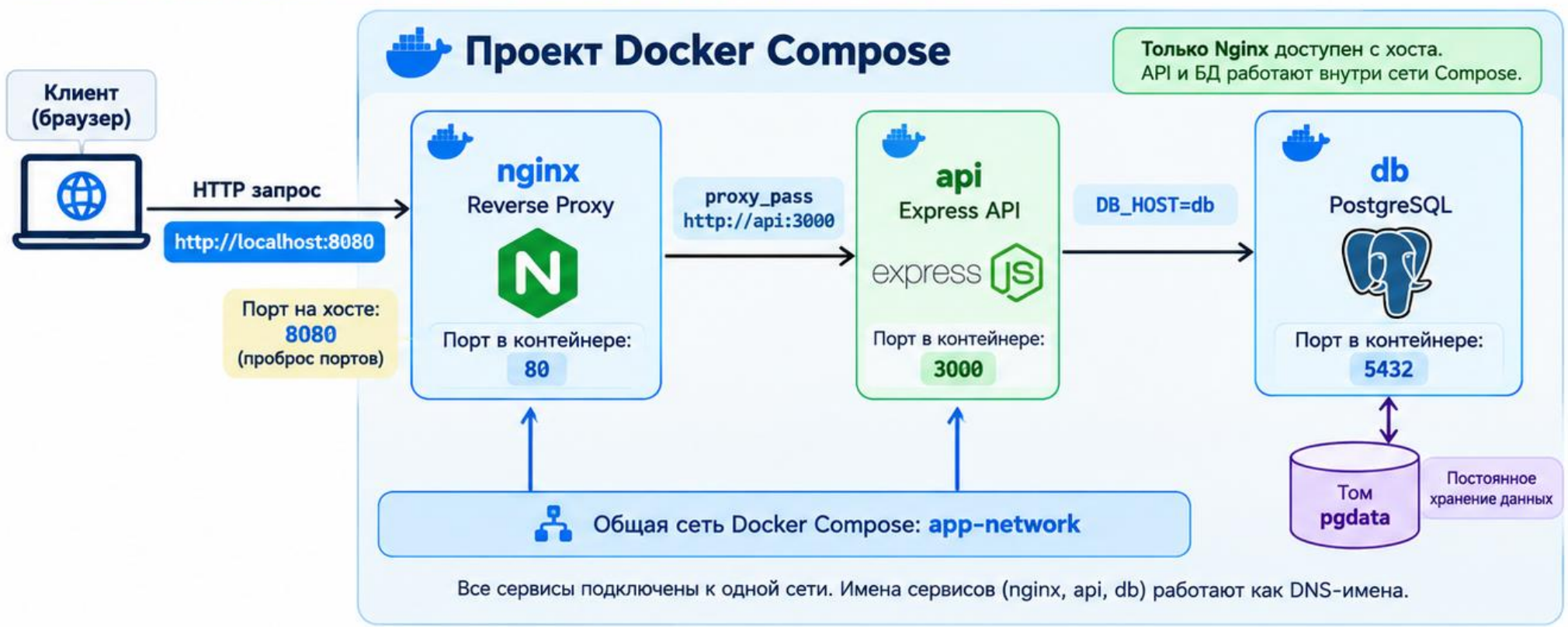
Как Compose объединяет несколько сервисов в одно приложение

compose.yaml

```yaml
services:
  nginx:
    image: nginx:latest
    ports:
      - "8080:80"
    networks:
      - app-network
  api:
    build: ./api
    networks:
      - app-network
  db:
    image: postgres:16
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - app-network

networks:
  app-network:

volumes:
  pgdata:
```



Что происходит

- **docker compose up** создаёт и запускает все сервисы
- **nginx** принимает запросы с хоста
- **api** обрабатывает бизнес-логику
- **db** хранит данные

Встроенный DNS

- **nginx** обращается к **api** по имени сервиса `proxy_pass http://api:3000`
- **api** обращается к **db** по имени сервиса `DB_HOST=db`
- **localhost** внутри контейнера не используют для связи между сервисами

Постоянные данные

- Том **pgdata** хранит данные PostgreSQL
- Данные сохраняются при пересоздании контейнера

Ключевые сервисы и порты

nginx	→	80	api	→	3000	db	→	5432	host (localhost)	→	8080

Роли сервисов

- nginx — reverse proxy; принимает HTTP-запросы клиента на порту 8080 хоста.
- api — Node.js + Express; выполняет HTTP-маршрут и обращается к БД.
- db — PostgreSQL 17; хранит данные в томе pgdata.
- Внутри Compose сервисы используют DNS-имена api и db.
- Публичный порт базы данных не нужен.

Nginx как reverse proxy

```nginx
server {
    listen 80;
    location / {
        proxy_pass http://api:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Что важно

- Nginx направляет HTTP-запросы сервису api по внутреннему адресу.
- api:3000 — имя Compose-сервиса и внутренний порт, а не localhost.
- В учебной конфигурации достаточно HTTP на локальном 8080.

Маршрут проверки PostgreSQL

```javascript
app.get("/api/db-check", async (req, res) => {
  try {
    const result = await pool.query("SELECT 1 AS co
    res.json({ ok: true, connected: result.rows[0].
  } catch (error) {
    res.status(503).json({ message: "БД недоступна"
  }
});
```

Что важно

- Запрос проходит клиент → Nginx → Express → PostgreSQL.
- Ответ 200 означает успешное SQL-подключение, а не только запуск контейнера.
- Ошибка 503 позволяет увидеть отказ базы данных.

Полный цикл лабораторного проекта

1	**Шаг 1**	Создать каталог с compose.yaml, исходниками API и nginx.conf.
2	**Шаг 2**	Подготовить Dockerfile и установить зависимости Express + pg.
3	**Шаг 3**	Проверить структуру файла командой docker compose config.
4	**Шаг 4**	Запустить nginx, api, db одной командой docker compose up -d --build.
5	**Шаг 5**	Проверить curl http://127.0.0.1:8080/api/db-check.
6	**Шаг 6**	Остановить и пересоздать стек без удаления данных; исследовать volume.

Типовые ошибки

- YAML: неправильный отступ нарушает вложенность.
- Nginx: proxy_pass http://localhost:3000 обращается не к API, а к Nginx-контейнеру.
- API: PGHOST=localhost вместо db приводит к ошибке соединения.
- БД: пароль или имя базы не совпадают с ранее инициализированным томом.
- Docker: порт 8080 занят другим процессом на хосте.

Правила безопасности для учебного стенда

- Публикуем 8080 только на 127.0.0.1 Debian, если внешний доступ не требуется.
- Не публикуем PostgreSQL 5432 на хост.
- Сохраняем исходный pg-course из ЛР №5 нетронутым: Compose использует отдельный том.
- Не используем down -v до завершения проверки сохранности данных.

Итоги лекции

1. Compose описывает несколько сервисов единым compose.yaml.
2. services задаёт конфигурацию, networks объединяет контейнеры, volumes хранит данные.
3. Контейнеры обращаются друг к другу по именам сервисов, а не по IP.
4. up -d --build создаёт стек; logs, ps, exec помогают диагностике.
5. down сохраняет именованные тома, а down -v может удалить учебную базу.
6. Схема клиент → Nginx → Express API → PostgreSQL станет основой следующей темы.

Контрольные вопросы

1. 1. Чем docker compose отличается от нескольких docker run?
2. 2. Зачем нужен compose.yaml?
3. 3. Чем build отличается от image?
4. 4. Почему API обращается к БД по имени db?
5. 5. Публикуется ли автоматически порт PostgreSQL для API?
6. 6. Чем отличаются volumes внутри сервиса и верхнеуровневый volumes?
7. 7. Что проверяет docker compose config?
8. 8. Чем down отличается от down -v?
9. 9. Гарантирует ли depends_on без healthcheck готовность БД?
10. 10. Почему proxy_pass указывает api:3000?